Heart Disease Risk Prediction

End-to-End MLOps: Model Development, CI/CD and Production Deployment

Course: Machine Learning Operations (AIMLCZG523) - Assignment 01

Student name / ID: 2025ae05858

Code repository: https://github.com/tanmayeebits5858/bits-heart-disease-mlops.git

Deployed API: Local Kubernetes (Minikube) behind a LoadBalancer Service via minikube tunnel; see Section 9
Video: https://github.com/tanmayeebits5858/bits-heart-disease-mlops/blob/671a37fdf24053c6a3cba923a47500cb0c882468/reports/MLOps_Assignment1_2025ae05858_Compressed_2.mp4

1. Summary

This project delivers a reproducible machine learning solution that predicts the presence of heart disease from 13 clinical features. The solution covers data acquisition, exploratory analysis, two tuned classifiers, MLflow experiment tracking, a packaged scikit-learn pipeline, a tested CI workflow, a Docker image serving a FastAPI endpoint, Kubernetes manifests, and Prometheus and Grafana monitoring.

Item	Result
Dataset	UCI Heart Disease (Cleveland), 303 patients, 6 missing values imputed, no duplicates
Selected model	Logistic Regression (C = 1, L2 penalty), chosen by highest mean cross-validated ROC-AUC
Cross-validated ROC-AUC	0.902 +/- 0.014 (5-fold, training split)
Hold-out test (61 patients)	Accuracy 0.885, precision 0.839, recall 0.929, ROC-AUC 0.966
Tests	19 unit tests passing; flake8 clean
Serving	FastAPI: /predict, /health, /metrics; 17 to 26 ms per prediction in local tests

Scope note. All stages were executed: data download, EDA, training, MLflow tracking, unit tests, lint, the GitHub Actions workflow (including an intentional failing run), the Docker build and run, the Kubernetes deployment on Minikube, and the Prometheus and Grafana stack. Screenshots in Sections 5 and 7 to 10 document these runs. All training results in this report come from the final run on the UCI data.

2. Setup and Installation

Requirements: Python 3.11 or later, Git, Docker, and optionally Minikube and kubectl.

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python scripts/download_data.py
export PYTHONPATH=src
python -m heartml.eda        # cleaning and figures
python -m heartml.train      # tuning, CV, MLflow logging, model packaging
pytest -v && flake8 src tests scripts
```

The download script tries the UCI repository first and falls back to a GitHub mirror of the cleaned CSV when UCI is unreachable. The final run used the UCI source directly. A clean virtual-environment install from requirements-dev.txt was verified: lint passes and all 19 tests pass.

3. Data Acquisition and Exploratory Data Analysis

3.1 Dataset and cleaning

The dataset contains 13 predictors and one target. Numeric features are age, resting blood pressure (trestbps), cholesterol (chol), maximum heart rate (thalach) and ST depression (oldpeak). Categorical features are sex, chest pain type (cp), fasting blood sugar (fbs), resting ECG (restecg), exercise-induced angina (exang), ST slope (slope), number of major vessels (ca) and thalassemia (thal).

- Duplicates: none. All 303 rows are retained.
- Missing values: 6 in the raw UCI file (ca: 4, thal: 2). They are read as NaN and imputed inside the pipeline (median for numeric, most frequent for categorical), so training and serving apply identical handling.
- Target: binarised to 0 (absence) and 1 (presence). UCI severity values 1 to 4 map to 1.
- Encoding: UCI codes are kept as downloaded (for example cp 1 to 4, slope 1 to 3, thal 3, 6 and 7) and treated as nominal categories by one-hot encoding, so no recoding is required.

3.2 Class balance

The classes are close to balanced: 164 patients without disease (54.1%) and 139 with disease (45.9%). Accuracy is therefore a valid metric, and recall is tracked because missed disease cases carry the highest cost.

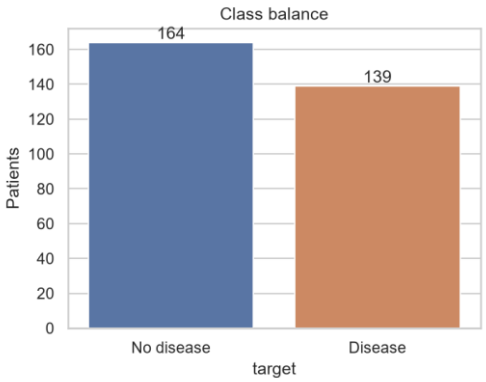


Figure 1. Class balance after cleaning

3.3 Distributions and relationships

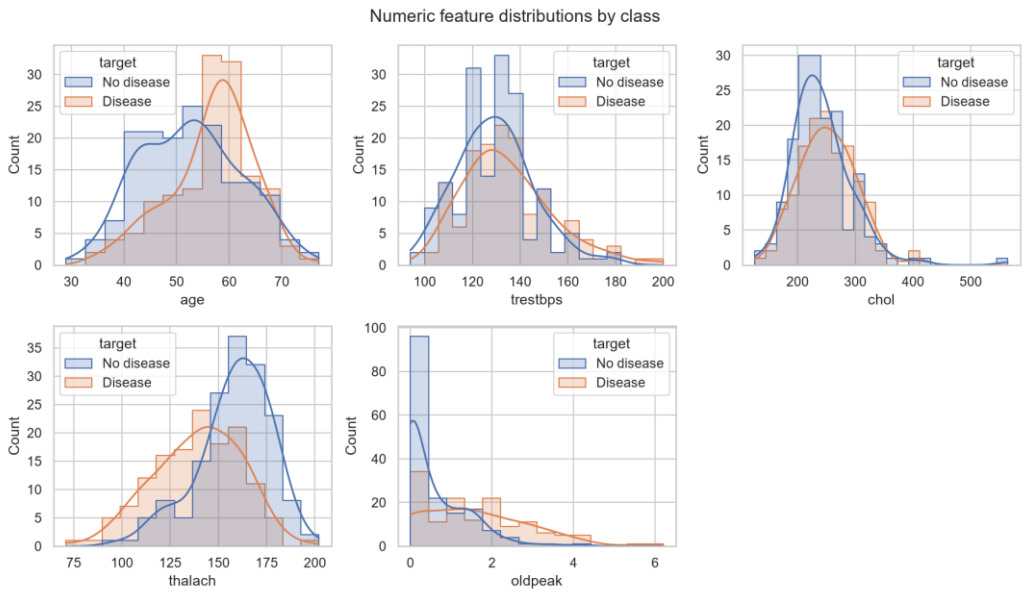


Figure 2. Numeric feature distributions by class

Patients with disease tend to show a lower maximum heart rate and higher ST depression. Age and resting blood pressure show weaker separation. Cholesterol has a long right tail with four values above 400 mg/dl (maximum 564).

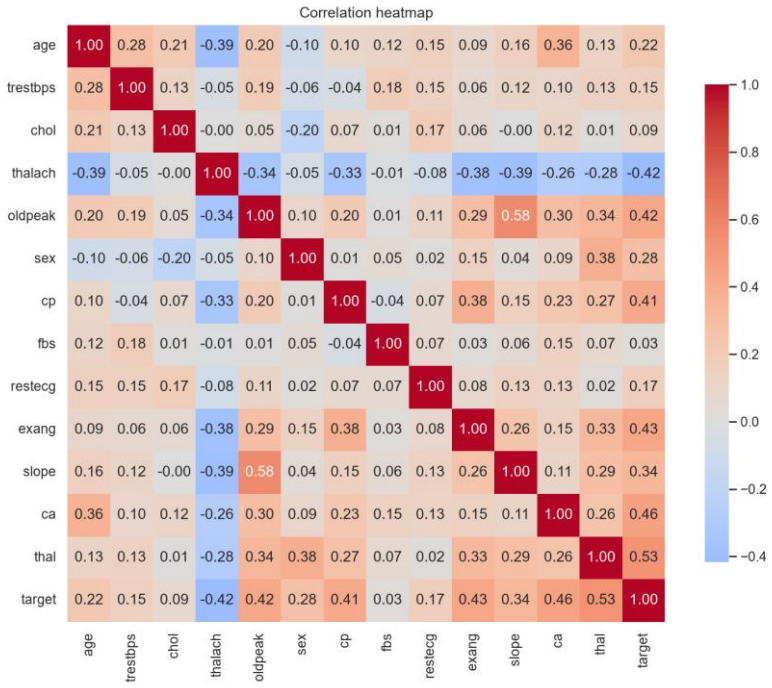


Figure 3. Correlation heatmap

The strongest correlations with the target are thalassemia (+0.53), number of major vessels (+0.46), exercise-induced angina (+0.43), ST depression (+0.43) and maximum heart rate (-0.42). No pair of predictors shows correlation high enough to require removal (maximum absolute pairwise correlation 0.58, between oldpeak and slope).

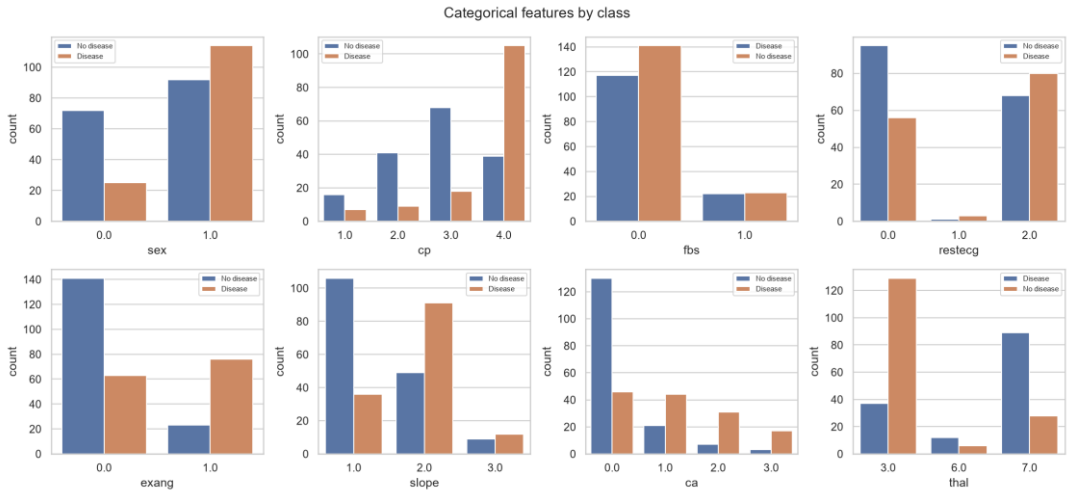


Figure 4. Categorical features by class

4. Feature Engineering and Model Development

4.1 Preprocessing pipeline

All preprocessing lives inside one scikit-learn Pipeline so that training and serving apply identical transformations.

Feature group	Steps	Rationale
Numeric (5)	Median imputation, StandardScaler	Scale-sensitive for logistic regression; robust to outliers via median
Categorical (8)	Most-frequent imputation, OneHotEncoder(handle_unknown='ignore')	Codes are nominal; unseen categories at inference do not raise errors

4.2 Models, tuning and evaluation protocol

Data was split 80/20 with stratification (242 training, 61 test rows, random_state 42). Each model was tuned with GridSearchCV using stratified 5-fold cross-validation and ROC-AUC as the objective. The tuned model was then re-evaluated with cross_validate on five metrics. The test split was used once, for final reporting, and not for selection.

Model	Search grid	Best parameters
Logistic Regression	C in {0.01, 0.1, 1, 10}, L2 penalty (4 candidates)	C = 1
Random Forest	n_estimators {100, 300}, max_depth {3, 5, None}, min_samples_leaf {1, 3} (12 candidates)	100 trees, max_depth 5, min_samples_leaf 3

Metric (mean +/- std, 5-fold CV)	Logistic Reg.	Random Forest	LR test	RF test
Accuracy	0.847 +/- 0.010	0.806 +/- 0.023	0.885	0.869
Precision	0.877 +/- 0.054	0.811 +/- 0.059	0.839	0.833
Recall	0.783 +/- 0.046	0.764 +/- 0.089	0.929	0.893
F1	0.825 +/- 0.007	0.781 +/- 0.032	0.881	0.862
ROC-AUC	0.902 +/- 0.014	0.901 +/- 0.024	0.966	0.947

4.3 Model selection and interpretation

The selection rule was fixed before training: highest mean cross-validated ROC-AUC. Logistic Regression wins by 0.002 (0.902 against 0.901), which is far smaller than the fold-to-fold standard deviations (0.014 and 0.024), so the two models are close in cross-validation on this criterion. Logistic Regression also leads on the other four cross-validated metrics and on all five hold-out metrics, although a hold-out set of 61 patients carries wide uncertainty. The result is consistent across both evaluations, so the simpler and more interpretable model is selected.

Recommendation: for a clinical deployment, validate on a larger external cohort and check probability calibration. The coefficients of Logistic Regression can be inspected directly, which supports clinical review.

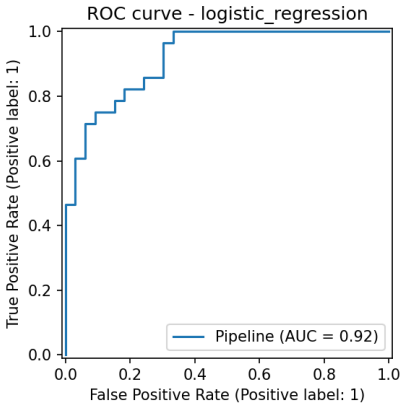


Figure 5. ROC curve, Logistic Regression (test split)

5. Experiment Tracking with MLflow

Every training execution logs to an MLflow experiment named heart-disease-classification, backed by SQLite (mlflow.db) with a local artifact store (mlartifacts/). Three runs are recorded per execution: one per model and one for the selected model.

Logged item	Content
Parameters	Best hyper-parameters, model name, CV folds, test size, random state, train and test sizes

Logged item	Content
Metrics	CV mean and std for accuracy, precision, recall, F1, ROC-AUC; test accuracy, precision, recall, F1, ROC-AUC
Artifacts	ROC curve, confusion matrix, grid_search_results.csv, serialized model with signature and input example, model metadata JSON

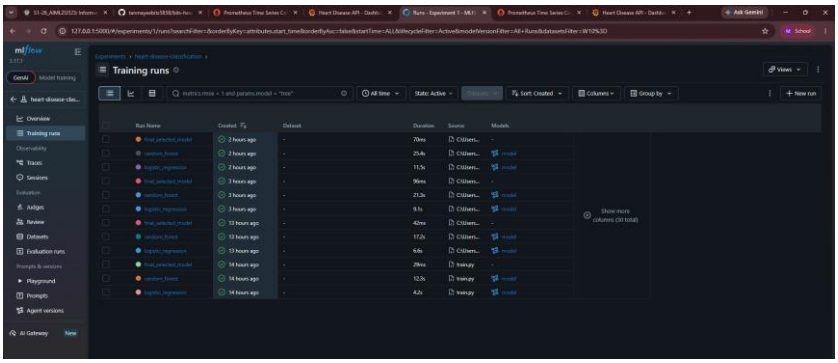


Figure 6. MLflow experiment: tracked runs, each with a logged model

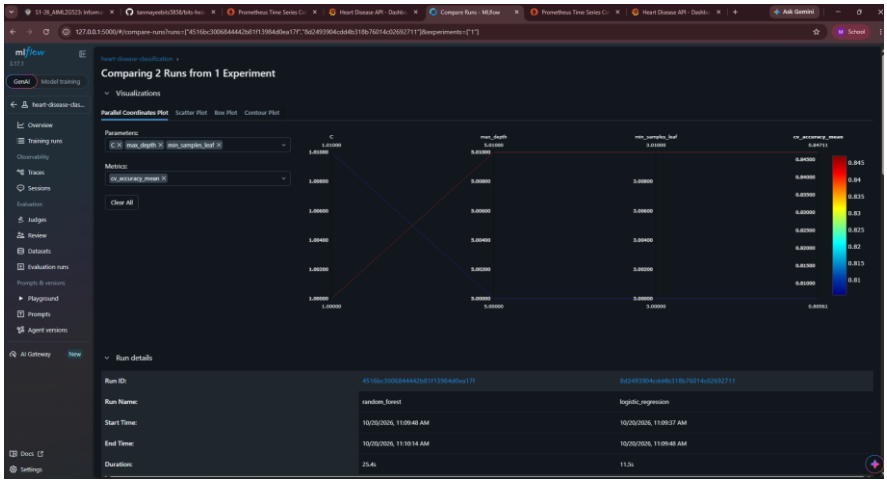


Figure 7. MLflow run comparison: Random Forest and Logistic Regression

Start the interface with: mlflow ui --backend-store-uri sqlite:///mlflow.db. If the interface renders a blank error page in an environment with a non-standard locale, start the server with --gunicorn-opts "--workers 1".

6. Model Packaging and Reproducibility

- **Model format:** the full Pipeline (preprocessing plus classifier) is saved with joblib as models/model.joblib and also logged to MLflow with cloudpickle serialization. A metadata file records the selected model, metrics, feature list and library versions.
- **Dependencies:** requirements.txt (training), requirements-dev.txt (testing and lint), and requirements-serve.txt (runtime, exact version pins). The serving scikit-learn version must match the training version (1.9.1 in the final run), because pickled pipelines are version sensitive.
- **Determinism:** fixed random_state 42 for the split, cross-validation and models. Preprocessing has a determinism unit test.
- **Clean setup:** a fresh virtual environment built from requirements-dev.txt passed flake8 and all 19 tests.

7. CI/CD Pipeline and Automated Testing

7.1 Unit tests

File	Tests	Coverage of behavior
tests/test_data.py	6	Target binarisation, duplicate removal, bad values to NaN, column order, schema validation, real file sanity

File	Tests	Coverage of behavior
tests/test_features.py	4	Finite and scaled output, imputation, unseen category handling, determinism
tests/test_model.py	4	Both models fit and return valid probabilities, metric set, quality floor (ROC-AUC above 0.8) on the saved model
tests/test_api.py	5	Health, prediction schema and confidence range, missing field (422), out-of-range value (422), metrics exposure

Result: 19 passed. Line coverage of the source package is 57% overall (API 92%, features 100%, data 90%); the training and EDA modules are exercised end to end by the CI training step rather than by unit tests. Raising unit coverage of train.py is a documented improvement.

7.2 GitHub Actions workflow

The workflow .github/workflows/ci.yml runs on every push to main and on pull requests. Any failing step stops the pipeline and the logs show the failing command.

#	Step	Behavior
1	Install	pip install -r requirements-dev.txt with pip cache
2	Lint	flake8 src tests scripts (max line length 100); fails on violation
3	Dataset	python scripts/download_data.py
4	Unit tests	pytest with coverage and JUnit XML
5	Train	python -m heartml.train, output captured with tee
6	Artifacts	Uploads junit.xml, coverage.xml, train_output.txt, models/, mlartifacts/, mlflow.db (runs even on failure)
7	Docker job	Builds the image, starts the container, calls /health and /predict, asserts a valid prediction, prints container logs

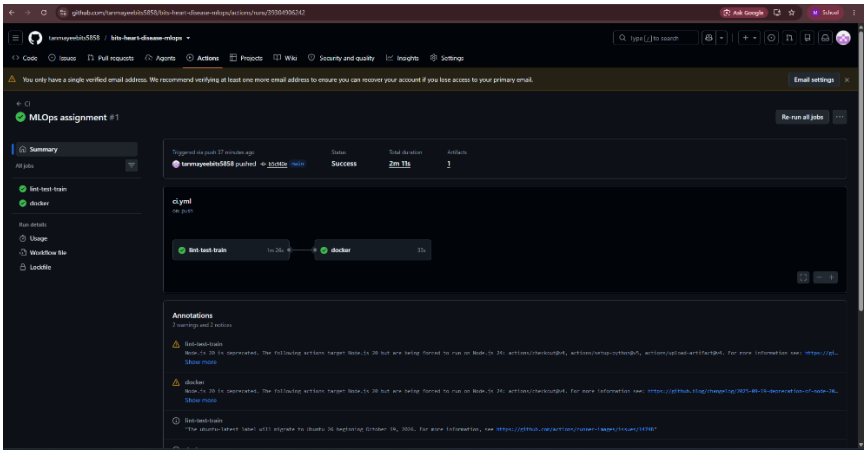


Figure 8. GitHub Actions: successful run with both jobs passing, and the ci-artifacts artifact

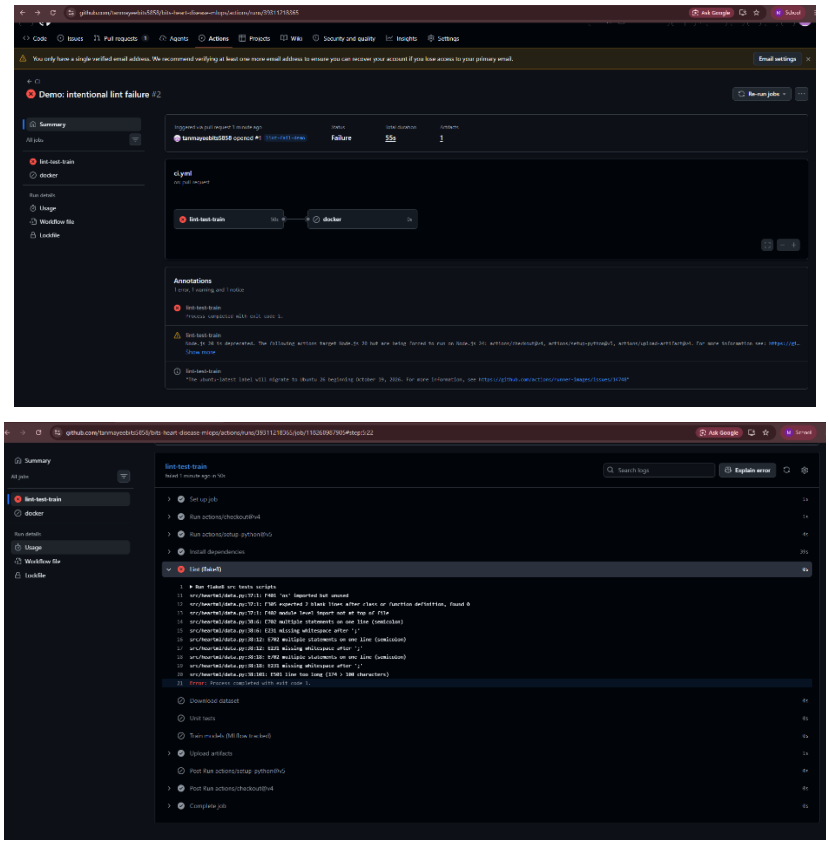


Figure 9. Intentional lint failure on a branch: the pipeline fails and the log identifies the error

Local evidence of the same steps is stored in the evidence folder (flake8.txt, pytest_output.txt, train_output.txt).

8. Model Containerization

The Dockerfile uses python:3.11-slim, installs only the pinned serving dependencies, copies the package and the packaged model, runs as a non-root user, and defines a HEALTHCHECK on /health.

```
docker build -t heart-api:1.0.0 .
docker run -p 8000:8000 heart-api:1.0.0
```

8.1 API contract

Endpoint	Description
POST /predict	Accepts JSON with the 13 features (validated ranges, 422 on invalid input). Returns prediction (0 or 1), label, confidence (probability of the predicted class), probability_disease and model_version.
GET /health	Returns 200 with model name and version when the model is loaded, 503 otherwise. Used by Docker and Kubernetes probes.
GET /metrics	Prometheus exposition format.

Verified request and response (executed against the running container):

```
POST /predict {"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,"fbs":1,"restecg":0,
               "thalach":150,"exang":0,"oldpeak":2.3,"slope":0,"ca":0,"thal":1}
200 {"prediction":0,"label":"no heart disease","confidence":0.8146,
   "probability_disease":0.1854,"model_version":"logistic_regression"}
POST /predict {"age":63}   ->   422 (validation error)
```



Figure 10. Docker container: API calls to /health and /predict

9. Production Deployment

Kubernetes manifests in k8s/ deploy two replicas with readiness and liveness probes on /health, resource requests and limits, and Prometheus scrape annotations. A Service of type LoadBalancer exposes port 80 to container port 8000. An optional Ingress routes heart-api.local through the NGINX ingress controller.

Step	Command
Start cluster	minikube start
Build image in cluster	eval $(minikube docker-env) && docker build -t heart-api:1.0.0 .
Deploy	kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
Expose LoadBalancer	minikube tunnel (separate terminal), then kubectl get svc heart-api
Verify	curl http://<EXTERNAL-IP>/health and POST to /predict
Optional Ingress	minikube addons enable ingress && kubectl apply -f k8s/ingress.yaml

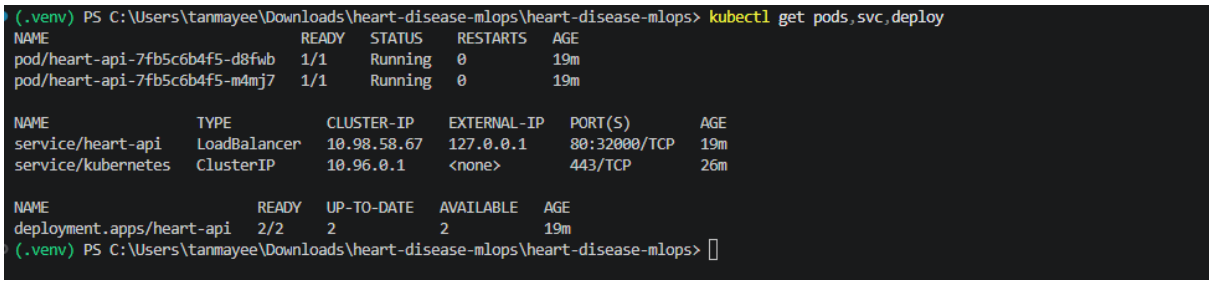


Figure 11. kubectl get pods,svc,deploy: 2 pods Running, LoadBalancer service with external IP



Figure 12. /health and /predict through the LoadBalancer address

10. Monitoring and Logging

10.1 Request logging

Every request produces one structured JSON log line with request id, method, path, status, latency in milliseconds and client address. Each prediction also logs the predicted class and confidence. Logs go to stdout (visible with docker logs and kubectl logs), and a rotating file is enabled with the LOG_FILE variable. Example log lines:

```
{"event": "request", "request_id": "29d2adb0", "method": "POST", "path": "/predict",
  "status": 200, "latency_ms": 25.62, "client": "127.0.0.1"}
{"event": "prediction", "prediction": 0, "confidence": 0.9741}
```

```
{"event": "request", "request_id": "6d15e021", "method": "POST", "path": "/predict",
  "status": 422, "latency_ms": 0.57, "client": "127.0.0.1"}
```


10.2 Prometheus and Grafana

Metric	Purpose
api_requests_total{method,path,status}	Request volume and error rate
api_request_latency_seconds (histogram)	Latency percentiles per path
model_predictions_total{prediction}	Class mix of predictions; shifts indicate input drift
model_confidence (histogram)	Distribution of prediction confidence
model_loaded	Model availability gauge

docker compose up --build starts the API, Prometheus (port 9090) and Grafana (port 3000). Grafana provisions the Prometheus data source and the Heart Disease API dashboard automatically. The dashboard has five panels: request rate by path and status, p95 latency, predictions by class, model loaded, and error rate. Verified locally: the /metrics endpoint returned api_requests_total, model_predictions_total and model_loaded with correct counts after test calls.

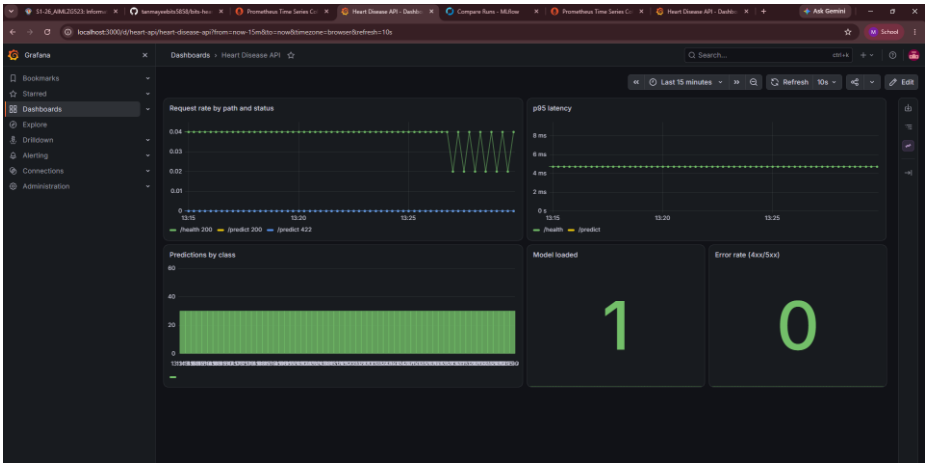


Figure 13. Grafana dashboard Heart Disease API after test traffic

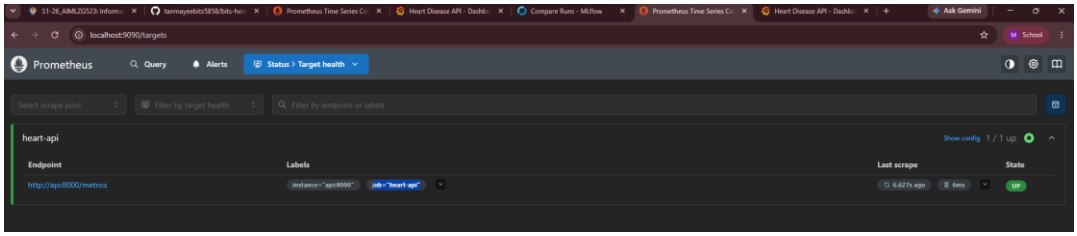


Figure 14. Prometheus targets: heart-api is UP

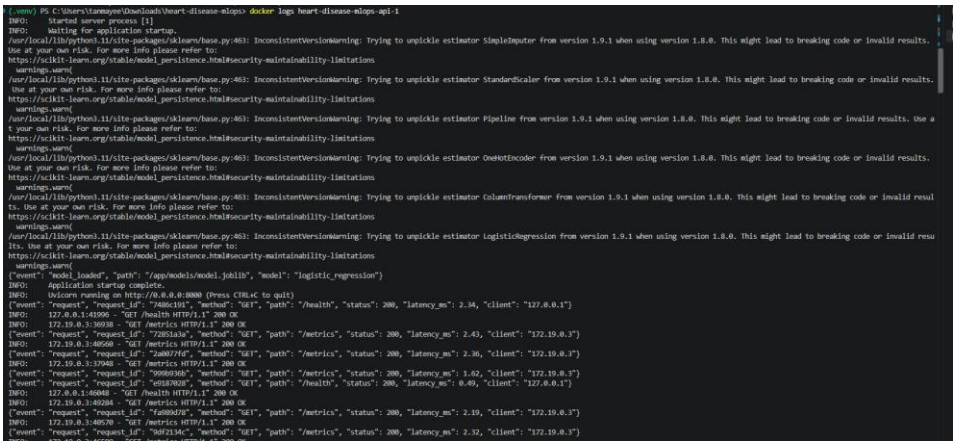


Figure 15. Container logs: structured JSON request records

11. Architecture

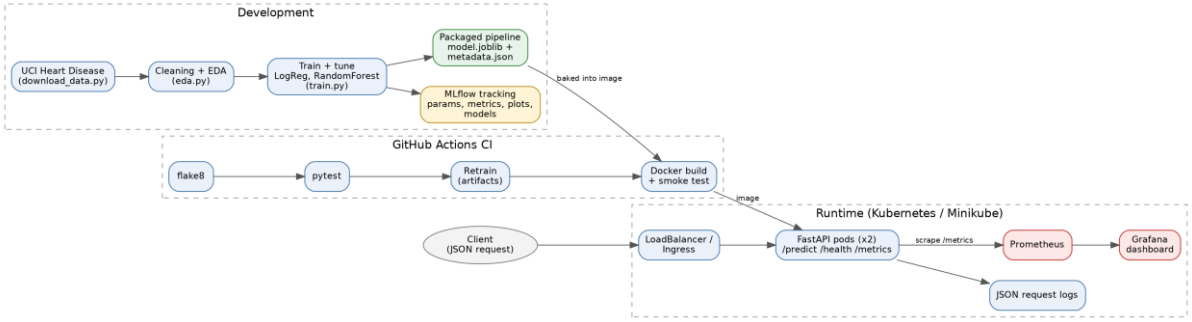


Figure 16. System architecture: development, CI, and runtime

Flow: data is downloaded and cleaned, models are tuned and tracked in MLflow, and the selected pipeline is saved. CI lints, tests, retrains and builds the image. The image runs as two Kubernetes pods behind a LoadBalancer. Prometheus scrapes /metrics and Grafana visualises the data.

12. Limitations and Recommended Improvements

- Small dataset (303 patients, 61 in the test split): metric estimates are noisy and the two models cannot be separated reliably on cross-validated ROC-AUC. Recommend repeated cross-validation and an external validation set.
- The dataset is not representative of a general population and the tool is for educational purposes only; it is not a medical device.
- Data handling: the raw UCI file contains six missing values, which are imputed with training-set statistics. The binary target merges severity levels 1 to 4; a multi-class severity model is not attempted.
- Model registry and promotion: the MLflow model registry is not used. Recommend registering the selected model and promoting by stage in a multi-environment setup.
- Test coverage of train.py is low (34%). Recommend adding tests with a small synthetic dataset for run_training.
- Monitoring covers service health and prediction mix. Recommend adding input drift statistics and alert rules in Prometheus.